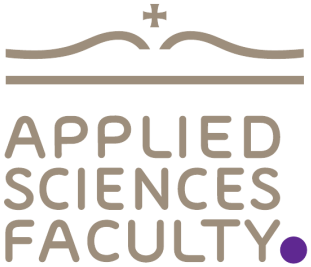
C++ — додаток. Qt. Lecture XX — багатопоточність





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

- Ідеологія багатопоточності Qt близька до ідеології POSIX Threads — pthreads та потоків C++11.
- Базовим класом для реалізації потоків є QThread.
- Найпростішим способом створити новий потік — успадкувати від QThread і перевизначити віртуальний метод **run()**.
- **Увага!** Такий метод найпростіший, але на практиці, зазвичай — поганий! Див. далі.
- Для запуску коду, реалізованого в run(), в іншому потоці, слід використати метод QThread::**start()**.
- **УВАГА!** Всі методи QThread, крім власне **run()** виконуються в контексті потоку, де знаходиться QThread.
- Інші важливі методи QThread: **quit()** та **exit(int returnCode = 0)** — завершують цикл обробки повідомлень потоку, повертаючи код завершення (нуль — для quit()).
- Чекати на завершення потоку, але не більше ніж вказаний час:

  bool QThread::wait (**unsigned long** time = ULONG_MAX)

  Час задається в мілісекундах (ULONG_MAX означає відсутність таймауту). Повертає true, якщо метод run() завершився (або не був запущеним на момент виклику), або false, якщо стався таймаут. Ця функція є аналогом pthread_join() та std::thread::join().
- Цикл обробки повідомлень в потоці запускається методом exec(),

- Об'єкти Qt пов'язані із потоками, у яких вони виконуються.
- Існують засоби перенесення об'єктів між потоками:

```
threadSource = new QThread();

WorkerSrc *worker = new WorkerSrc();
worker->moveToThread(threadSource);
```

- Видаляти нащадків QObject із інших потоків (чи, у загальному випадку, взагалі звертатися до них таким чином) заборонено. Єдиний виняток, коли це можна робити — є впевненість, що QObject не зайнятий обробкою повідомлень. В іншому випадку для видалення краще скористатися QObject::deleteLater(), який пошле повідомлення DeferredDelete. Тобто, робота deleteLater() теж залежить від наявності потоку обробки повідомлень у даному потоці (див. нижче).

Види методів Qt по відношенню до багатопоточності:

- Реентрантні методи (Reentrant) — методи, які можуть бути одночасно викликані з різних потоків, якщо вони працюють над **різними** об'єктами. Тобто, це методи, які не працюють із глобальними чи статичними даними. Однак, вони не є атомарними, а тому не є потоко-безпечними.
- Потоко-безпечні методи (thread-safe) — методи, які можна безпечно викликати із різних потоків, навіть якщо вони працюють із тими ж самими даними. Тобто, звертання до даних серіалізуються (із використанням примітивів синхронізації – мютексів, семафорів і т.д.)
- Решта методів не можна викликати із різних потоків одночасно.

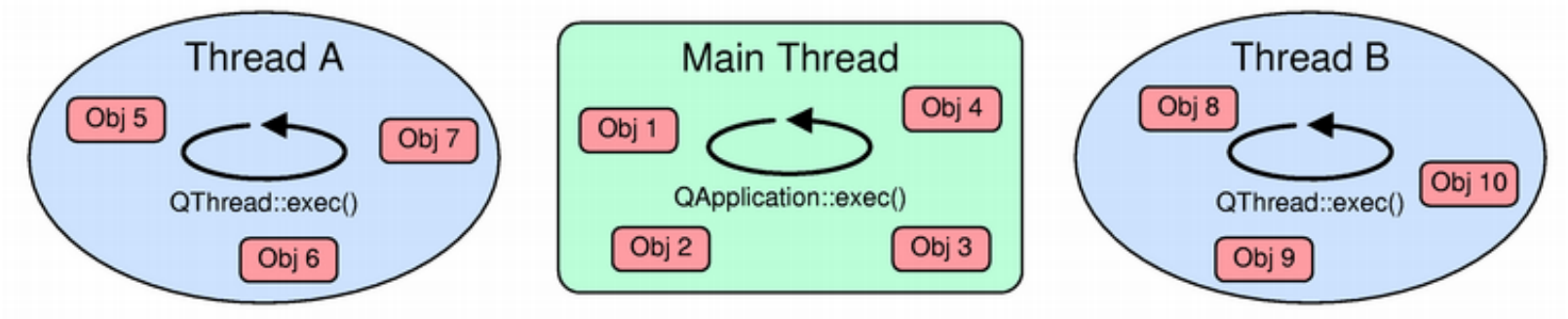
UI Thread

В Qt працювати із GUI може тільки головний потік!

- QCoreApplication можна викликати лише із головного потоку.
- QWidget і його нащадки **не є реєнтрантними**.
- Також нереєнтрантними є деякі пов'язані із QWidget класи, такі як QPixmap.
- Сам QObject — реєнтрантний. Однак, всі його "діти" (в сенсі композиції а не успадкування) повинні належати одному потокові.

Цикли обробки повідомлень в потоках

- Кожен потік може мати свій цикл обробки повідомлень. Запускається він викликом методу exec().
- Наявність потоку обробки повідомлень дає можливість нормально функціювати таким класам, QTimer, QTcpSocket, QProcess.
- З потоком, що має цикл обробки повідомлень, можна вільно обмінюватися повідомленням та сигналами.



Зображення взято з офіційної документації Qt.

Альтернативи до багатопоточності в Qt

- Використання кількох процесів, які обмінюються повідомленнями — pipes або технології типу OpenMPI. (Shared memory має більшість основних проблеми багатопоточності, тому вона не про нього!)
- QEventLoop::processEvents() — виклик "примусової" обробки повідомлень під час тривалих операцій, щоб не блокувати потік GUI.
- QTimer, які дозволяють викликати вказані функції (слоти) із потрібним інтервалом, або один раз в потрібний момент. Додатково є можливість виклику, коли завершилися повідомлення для обробки.
- QSocketNotifier, для повідомляння про активність ОС-специфічних файлових дескрипторів, QNetworkAccessManager — для асинхронної роботи із мережею, QIODevice::readyRead() для асинхронної роботи з нащадками QIODevice
- Високорівневий інтерфейс, **QtConcurrent**, який дозволяє розподіляти наявні ресурси між складними обчислювальними задачами, інкапсулюючи низькорівнені маніпуляції із QThread.

Синхронізація

Qt надає стандартний набір засобів синхронізації багатопоточних задач.

- QMutex — реалізує примітив синхронізації mutex, mutual exclusion.
- QMutexLocker — Допоміжний клас, конструктор якого блокує QMutex, а деструктор – розблоковує.
- QReadWriteLock — клас блокування, який допускає одночасне читання даних, поки не відбувається запис.
- QReadLocker і QWriteLocker — відповідні допоміжні класи, аналогічні до QMutexLocker.
- QSemaphore — узагальнення мютекса, для захисту кількох однакових ресурсів.
- QWaitCondition — дозволяє синхронізувати потоки не блокуючи їх виконання, а навпаки, пробуджуючи, коли виконується певна умова.
- Шаблони класів QAtomicInteger[1] та QAtomicPointer надають інтерфейс атомарних змінних. Для типів, атомарні операції над якими підтримуються процесором, Qt намагається реалізовувати ці операції без блокувань (lock-free чи wait-free в термінах Qt).

[1] Включаючи QAtomicInt, який є просто QAtomicInteger<int>



Thread-local storage



QThreadPool і QRunnable





Високорівневі асинхронні інтерфейси Qt — QtConcurrent

-





Посилання

- http://qt-project.org/doc/qt-5/thread-basics.html — Threading Basics.
- http://qt-project.org/doc/qt-5/qthread.html — QThread Class.
- http://qt-project.org/doc/qt-5/threads-technologies.html — Multithreading Technologies in Qt.
- http://qt-project.org/doc/qt-5/threads-synchronizing.html — Synchronizing Threads
- http://qt-project.org/doc/qt-5/threads-reentrancy.html — Reentrancy and Thread-Safety.
- http://qt-project.org/doc/qt-5/threads-qobject.html — Threads and QObjects.
- http://qt-project.org/doc/qt-5/qtconcurrent-index.html
- http://qt-project.org/doc/qt-5/examples-threadandconcurrent.html — Threading and Concurrent Programming Examples.

Посилання – Concurrency, TLS, інше

`http://blog.qt.digia.com/blog/2007/04/26/mapreduce-in-qt-concurrent/` — "MapReduce in Qt Concurrent"
`http://www.bogotobogo.com/Qt/Qt5_QtConcurrent_QFutureWatcher_QProgressDialog_map.php` — Qt5 Tutorial QtConcurrent QProgressDialog with QFutureWatcher

- `http://qt-project.org/doc/qt-5/qthreadstorage.html` — QThreadStorage Class.
- `http://habrahabr.ru/post/150801/` — Немного о многопоточном программировании. Часть 1. Синхронизация зло или все-таки нет.

Посилання – атомарність, lock-free

- http://qt-project.org/doc/qt-5/qatomicinteger.html — QAtomicInteger Class, http://qt-project.org/doc/qt-5/qatomicint.html — "QAtomicInt Class Reference", http://qt-project.org/doc/qt-5/qatomicpointer.html, http://qt-project.org/doc/qt-4.8/atomic-operations.html — "Implementing Atomic Operations"

Приклади:

- http://qt-project.org/doc/qt-5/qtcore-semaphores-example.html – Semaphores Example.
- http://qt-project.org/doc/qt-5/qtcore-waitconditions-example.html – Wait Conditions Example.
- http://www.bogotobogo.com/Qt/Qt5_QThreads_QSemaphore_Producer_Consumer_GuiThread.php — ілюстрація на тему "QThreads and QSemaphore for Producer and Consumer".

Альтернативи до багатопоточності:

- http://doc.qt.digia.com/qq/qq11-events.htm — "Another Look at Events".
- http://qt-project.org/doc/qt-5/qtimer.html — QTimer Class
- http://qt-project.org/doc/qt-5/qsocketnotifier.html — QSocketNotifier class.
- http://qt-project.org/doc/qt-5/qnetworkaccessmanager.html — QNetworkAccessManager class.
- http://qt-project.org/doc/qt-5/qiodevice.html#readyRead — сигнал QIODevice::readyRead().





























